Gerard López Golfe

Case study | SecretarIA, a self-hosted RAG assistant in production

Vilamarxant (Valencia), Spain | linkedin.com/in/gerard-skt
Code and decision log: github.com/SureKT/secretaria-rag

THE PROBLEM

The administration offices of two public vocational schools in Valencia (CIPFP Ausiàs March and CIPFP Mislata) answer the same questions all year: courses, admissions, grants, timetables, procedures. They wanted an assistant on their websites, with one hard constraint: no student or staff query could leave the building.

WHAT I BUILT (ON MY OWN, MAR – JUL 2026)

- Scoping with the schools, ingestion, retrieval, the agent, the evaluation suite and deployment.
- Website widget → FastAPI service that streams the answer token by token over server-sent events.
- Retrieval: mxbai-embed-large embeddings in Qdrant, hierarchical chunking, and a per-school filter on every search (multi-tenant), plus content shared by both schools.
- Generation with a local LLM on Ollama, on the institute's NVIDIA DGX Spark. Conversation memory in PostgreSQL; Docker Compose and nginx. 25 tests with fakes for Qdrant, Ollama and PostgreSQL.

Result: phone calls to the office dropped to about a quarter within a few weeks.

THREE DECISIONS

- **I replaced my own first version.** It ran on n8n, which only returns complete responses, and users read several seconds of silence as a broken bot. I rewrote the backend in FastAPI to stream tokens: same model, same answers, and the problem was gone.
- **Wrong answers were not the model's fault.** Questions about one course came back with another course's content. I built a retrieval audit (retrieved chunks and their scores, no LLM) to tell retrieval failures from model failures. The cause: full-text search with no accent folding. Retrieval moved to embeddings.
- **One index for two schools.** Common content is stored once; isolation depends on a tenant filter that is built in a single function and checked by tests. The options I discarded are in an architecture decision record (ADR).

EVALUATION

The production model was chosen from a 36-case suite, not from impressions: questions per course family, timetables and procedures, **negative cases** where the right answer is to decline and redirect, and **multi-turn chains** that test conversational memory. Each case records pass/fail and latency. The suite also caught false positives in its own assertions, which I fixed.

LIMITS, AND WHAT I'D DO NEXT

- No reranking yet: a cross-encoder pass after retrieval is the obvious next step.
- Assertions are keyword-based, not model-graded, and there is no online feedback from real use.

Stack: Python, FastAPI, Qdrant, Ollama (mistral-small3.2), mxbai-embed-large, PostgreSQL, Docker Compose, nginx, n8n (first version).